

Building **Agentic AI**

Day 1 · Python for AI & the shape of data

Alex Gherghina-Pestrea · Radu Marin · Radu Ciobanu

Google Lab EG302 · August 2026

Before we start

- 📦 Clone the repo: <https://github.com/alexandru-g/google-agentic-summer-school>
- ⚙️ Follow `README.md` → `venv`, `pip install`
- 🔑 Then grab your API key → next slide

Get your free Gemini API key (~3 min)

1. Sign in at aistudio.google.com with a **personal Google account**
2. Click **Get API key** → **Create API key** (a new project is fine)
3. In the repo root: `cp .env.example .env`, paste the key in:
`GOOGLE_API_KEY=AIza...`
4. Run the smoke test from `README.md`. It prints **"Ready."**
 - The **free tier is enough for Days 1–3**: the code caches and retries around its rate limits
 - Keep it secret: `.env` is gitignored, leave it that way
 - **Day 4** you'll switch to Google-provided keys for Days 4–5 (same file, new value)

The week

Day	Theme	You build
1	Python for AI & the shape of data	Search + extraction, by hand
2	Your first agent	The model calls <i>your</i> code
3	Memory, state & real tools	Sessions + RAG + guardrails
4	Multi-agent systems	Orchestrator + specialists
5	Trust, evaluation & Demo Day	Proof it works, then demo

Your job this week 🎮

You are the new **data team of Playfield**, a small indie game storefront.

- **20 games · 300 player reviews** · store pages · patch notes
- The studios keep asking: *"What are players actually saying?"*

Small print: **we invented every one of these games.**

Remember that detail. It becomes important in about 40 minutes.

Today, in four parts

1 • Talking to a token generator	first API calls
2 • Meaning as geometry	vectors & embeddings
3 • Semantic search	300 reviews, 15 lines
4 • Messy text → clean rows	the LLM as a data machine

Part 1

Talking to a **token generator**

What is ChatGPT / Gemini / Claude, really?

Strip away the chat window and there is exactly one function:

```
      +-----+
text --->|   model   |---> a bit more text
      +-----+
```

- No database of facts
- No goals, no plan, no memory
- Just: **"given this text, what token probably comes next?"**

The loop that generates everything

1. Your prompt is cut into **tokens** (chunks of characters)
2. The model scores **every possible next token** with a probability
3. One token is picked and appended
4. Go to step 2, until a *stop* token wins

"The game crashes after the " → { update: 31%, patch: 24%, race: 8%, ... }

Every essay, poem, and answer you've ever seen from an LLM:
this loop, on repeat.

Tokens, not words

Text	Tokens
Hello	1
The game crashes after the update.	~7
Jocul se blochează după actualizare.	~11
   	~8

- Same meaning in Romanian $\approx$ **1.5× the tokens** (tokenizers are English-biased)
- You pay per token · context windows are measured in tokens
- Token math becomes real engineering on Day 5

"But it knows things!"

It behaves as if it does, because patterns in its training data encode facts.

- "The capital of France is ____" → Paris is simply the likeliest token
- There is **no lookup**, no citation, no checking
- When the pattern is missing... the loop **keeps generating anyway**

Plausible is the objective. True is a happy accident.

Temperature: the chaos dial

The model outputs a *probability distribution*. Temperature reshapes it:

0.0	always the top token	repeatable, focused
1.0	sample proportionally	varied, creative
2.0	flatten it	chaos gremlin

Rule for this course: **agents run cool.**

Tool decisions need reliability, not poetry.

The model has no memory

Two separate API calls:

```
call 1: "Hi, my name is Ana!"      → "Nice to meet you, Ana!"  
call 2: "What is my name?"        → "I don't have access to..."
```

Every chat app you've used fakes memory the same way:
re-send the entire history with every message.

📌 On Day 3 this becomes *sessions & state*: memory is a property of the **system**, not the model.

Notebook: `part1_llms.ipynb`

Sections 1–5, code-along:

-  first API call
-  count tokens (EN vs RO)
-  watch it stream, token by token
-  temperature 0 → 2
-  prove statelessness, then fake memory with `chats`

The experiment you just set up

Our catalog is fictional. So let's ask Gemini:

| *"Who developed **Neon Drift Racers** and when was it released?"*

Predictions?

Hallucination: the loop never says "segfault"

Two outcomes, both instructive:

- **A confident, specific, wrong answer:** plausible tokens, zero truth. That's a *hallucination*. It cannot be patched away.
- **"I'm not sure about that game":** not knowledge either! Post-training taught the model *when to fold*.

The fix is never "hope harder":

- **Give it the facts** → context, retrieval (today & Day 3)
- **Give it tools** → let it look things up (next session)

What you now know

- LLM = **next-token predictor**, nothing more, nothing less
- Tokens cost money and space; Romanian costs ~1.5×
- Temperature trades reliability for creativity
- **Stateless**: memory is an illusion built by re-sending
- **Hallucination** is the default failure mode; *grounding* is the fix

Exercises 7.1–7.4, especially **7.3 (JSON by politeness)**.

Keep your score. It comes back this afternoon.



Break

10 minutes

Part 2

Meaning as **geometry**

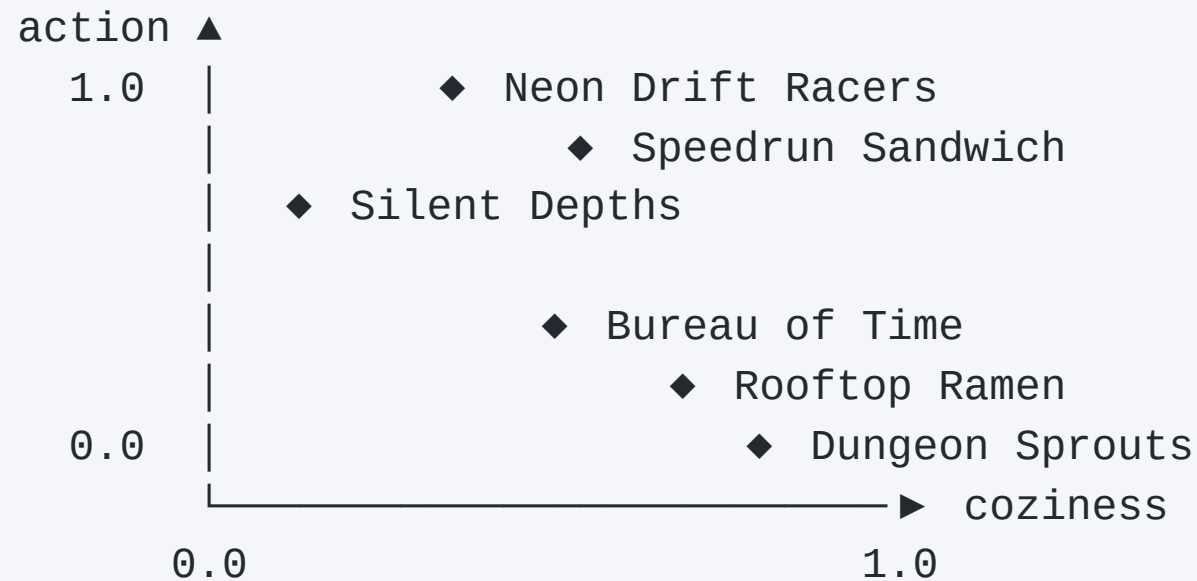
The problem

Computers compare **numbers**, not meanings.

- "game keeps freezing" vs "constant crashes": same complaint, zero shared words
- `str.contains` sees nothing

Idea: turn text into *coordinates*, so that
similar meaning = nearby points.

A toy meaning-space (2 axes, by hand)



Similar *vibe* → close together. Distance is now **computable**.

From dot product to cosine

Dot product: multiply element-wise, sum. Big = pointing the same way.

Flaw: longer vectors always win. Fix: compare **direction only**:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|} \in [-1, 1]$$

- **+1** same meaning · **0** unrelated · **-1** opposite
- Four lines of numpy. You'll write it from scratch.

Embeddings: the real thing

A trained model places **any text** in a ~768-dimensional meaning space.

- Axes are **learned**, not designed: no axis "means" anything
- Only **distances** matter
- Trained on many languages → **one space for all of them**

```
client.models.embed_content(  
    model="gemini-embedding-001",  
    contents=["The soundtrack is gorgeous.", "Muzica e superbă."],  
)  
# → two nearby points. Different languages. Same meaning.
```

One detail that matters later

Embedding models accept a **task type** hint:

task_type	for
SEMANTIC_SIMILARITY	comparing sentences to each other
RETRIEVAL_DOCUMENT	corpus entries (the <i>answers</i>)
RETRIEVAL_QUERY	searches (the <i>questions</i>)

Questions and answers are *phrased* differently, yet the model maps them close together **if you label which is which**.



Notebook: `part2_vectors.ipynb`

-  numpy speedrun (arrays, `argsort` , remember it!)
-  `cosine()` from scratch
-  embed 8 sentences → similarity heatmap
-  the multilingual reveal  ↔ 
- exercises: odd-one-out, vectorized cosine matrix



Lunch

30 minutes

Afternoon: we stop playing with 8 sentences
and search **all 300 reviews**.

Part 3

Semantic **search**: 300 reviews, 15 lines

Playfield needs answers

The studios ask things like:

- *"Are players still hitting crashes after our patch?"*
- *"What do people say about our monetization?"*
- *"Find me the reviews where someone cried."*

First instinct: `df[df.review_text.str.contains("crash")]`

Keyword search is a spelling matcher

Query	Hits	Reality
crash	✓ some	misses "CTD", "freezes", "hangs", "dies after update"
game keeps freezing	✗ 0	reviews say it 10 different ways
predatory monetization	✗ 0	reviews say "wallet warriors", "pay-to-win"
se blochează 🇷🇴	✗ 0	corpus is (mostly) English

Players don't use your words. **Meaning search or bust.**

The whole architecture

1. **Embed every review once** → matrix of 300×768 numbers → *cache it*
2. **Embed the query** (as `RETRIEVAL_QUERY`)
3. **One matrix multiply** → cosine score against every review at once
4. `argsort` , **take top-k** → done

That's it. That's the "R" in **RAG**.

~15 lines of Python. No vector database required at this scale.

The 3 habits that make it production-ish

- **Batching:** the API takes ~100 texts per call, not 300
- **Caching:** embeddings never change; compute once, `np.save`
- **Retries with backoff:** rate limits (429) are a *when*, not an *if*

These three show up in every real AI system you will ever build, and in your agent's tools from Day 3 on.



Notebook: `part3_search.ipynb`

- ✓ meet the corpus (pandas: `merge` , `groupby` , `value_counts`)
- ✓ watch keyword search fail
- ✓ embed 300 reviews (batched · cached · retried)
- ✓ write `search()` : the 15 lines
- ✓ interrogate it: crashes, monetization, tears, **Romanian queries**
- exercises: scoped search, most-similar pair, honest thresholds



Break

10 minutes

One part left: the LLM becomes a **data machine**.

Part 4

From **messy text** to **clean rows**

The question pandas can't answer

"What are the top complaints per game?"

That's a `groupby` ... on columns that **don't exist**:

- sentiment: trapped in prose
- issue type: trapped in prose
- sarcasm: *"10/10 would fall through the floor again"* 🙄

Our job: **make them columns.**

Attempt 1: ask nicely for JSON

```
"Reply ONLY with JSON: {\\"sentiment\\": ..., \\"issues\\": [...]}"
```

Exercise 7.3 scores, anyone?

- ````json` fences · friendly preambles · `"issues": "none really"`
- Works ~4 times out of 5. **Pipelines need 5 out of 5.**

Attempt 2: a contract, not a request

```
class ReviewAnalysis(BaseModel):
    sentiment: Literal["positive", "negative", "mixed"]
    issues: list[Issue]          # an Enum, groupby-able!
    is_sarcastic: bool
    summary: str

response = client.models.generate_content(
    model=MODEL, contents=review,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=ReviewAnalysis,    # ← the contract
    ),
)
response.parsed    # → a typed ReviewAnalysis object. Every time.
```

Why this works: constrained decoding

The schema doesn't *ask* the model to behave:
it **masks the tokens** that would violate the format.

- Invalid JSON becomes *impossible to emit*, not impolite
- Enums beat free text: `crashes_or_bugs` , not `"the game is crashy"`
- Structured output is how agents will pass data to each other on Day 4

Prompting is negotiation. Schemas are law.



Notebook: `part4_extract.ipynb`

- ✓ the sarcastic review pandas can't read
- ✓ JSON-by-politeness, one last time ✨
- ✓ `response_schema` → typed objects
- ✓ extract 40 reviews → **the complaints-per-game chart**
- ✓ find the reviews where the thumbs **lie**

The thumbs lie

recommended = ✓ but sentiment = negative

"437 hours. Do NOT buy. I can't stop. Send help."

- Labels and text disagree: extraction sees what `value_counts` can't
- And the model's `is_sarcastic` flag? **Check it. It misses some.**
- Model output is a *judgment*, not ground truth
📌 that thought is Day 5's entire topic (evaluation)

What you actually built today



This pipeline *is* Day 5's multi-agent system.

Today, *you* decided every step, in order, by hand.

Next session: the model starts deciding

```
# Day 2: Google's Agent Development Kit
from google.adk.agents import LlmAgent

analyst = LlmAgent(
    model="gemini-2.5-flash",
    name="playfield_analyst",
    instruction="Answer questions about Playfield's games using your tools.",
    tools=[search_reviews, analyze_review],  # ← TODAY's functions, wrapped
)
```

Same model. Same functions. One new ingredient:

the model reads your goal and chooses which tool to call.

That leap, from *pipeline* to *agent*, is where the next session starts.

Before you leave

- All four notebooks run top to bottom (re-run if anything errored)
- `day1/cache/` exists (the Day-2 scaffold assumes it)
- You can explain to a rubber duck *token, temperature, statelessness, hallucination,
embedding, cosine, task type, response schema*

See you next time

Day 2: **Your first agent**

The model calls your code.

Questions? Stay. Mentors are around.